Sistema de Gerenciamento e Planejamento de Missões Espaciais

Aluno: Leandro Castro Delevati
Linguagem: Python

1. Fonte de Dados

Item	Descrição
Nome da API	The Solar System OpenData
Endereço base	https://api.le-systeme-solaire.net/rest/bodies
Autenticação	Bearer Token, lido da variável de ambiente SOLAR_API_KEY
Formato de retorno	JSON, lista de objetos sob a chave "bodies"
Data de consulta	30/09/2026

Justificativa da escolha:

A API oferece uma quantidade grande e diversa de corpos celestes, cada um com atributos físicos e orbitais suficientes para dar suporte às operações de busca, pesquisa, filtragem e ao algoritmo guloso de planejamento de missão.

Endpoint utilizado:

```
GET /rest/bodies
Headers: Authorization: Bearer <SOLAR_API_KEY>
```

Exemplo de um item retornado:

```
{
  "id": "terre",
  "englishName": "Earth",
  "bodyType": "Planet",
  "isPlanet": true,
  "semimajorAxis": 149598023,
  "mass": { "massValue": 5.9722, "massExponent": 24 },
  "gravity": 9.8,
  "meanRadius": 6371.0,
  "avgTemp": 288,
  "moons": [ { "moon": "Moon", "rel": "..." } ],
  "discoveredBy": "",
  "discoveryDate": ""
}
```

- Faz GET real na API em tempo de execução, usando requests, e converte o JSON retornado em objetos CorpoCeleste. (Também criei uma opção para ler diretamente de arquivo local)
- **pip install -r requirements.txt** (Dependência para fazer requisições)
- Você precisa da própia chave API se quiser fazer requests:
  - export SOLAR_API_KEY='seu-token-aqui' (Bash)
  - $env:SOLAR_API_KEY = "seu-token-aqui" (Powershell)

2. Modelagem

Entidade: CorpoCeleste
Local: src/models/corpo_celeste.py

O papel desta classe é isolar o restante do sistema do formato bruto da API, se a API mudar um nome de campo só este arquivo precisa ser ajustado. A conversão é centralizada em api_dict, tanto para o retorno da API quanto para o arquivo .json local, os dois níveis de aquisição convergem para o mesmo formato interno.

Atributo	Tipo	Origem no JSON	Descrição
id	str	id	Identificador único usado como chave na Tabela Hash
nome	str	englishName	Nome em inglês, usado nas pesquisas
tipo	str	bodyType	Star, Planet, Dwarf Planet, Asteroid, Comet, Moon
e_planeta	bool	isPlanet	Flag booleana
distancia_km	float	semimajorAxis	Distância média ao corpo central
massa_kg	float	mass.massValue * 10^massExponent	Já calculada no api_dict
gravidade	float	gravity	Gravidade superficial (m/s²)
raio_medio_km	float	meanRadius	Raio médio
temperatura_media_k	float	avgTemp	Temperatura média (K)
quantidade_luas	int	len(moons)	0 se o campo não existir
descoberto_por	Optional[str]	discoveredBy	Opcional
data_descoberta	Optional[str]	discoveryDate	Opcional

3. Estrutura de dados

Interface: IEstruturaBusca
Local: src/estruturas/interfaces.py

IEstruturaBusca é uma classe abstrata que define o contrato mínimo que **qualquer** estrutura deve seguir, independentemente do seu funcionamento interno. Isso significa que, na Parte 2, ao implementar Trie e Árvore B, apenas fiz elas herdarem de IEstruturaBusca, assim como a Tabela Hash já herda, e implementei os mesmos seis métodos. O restante do sistema não precisará ser alterado.

Métodos declarados:

Todos os métodos são marcados com @abstractmethod, o que impede a instanciação de qualquer subclasse que não os implemente todos, Python levanta TypeError em tempo de execução se isso for tentado.

Método	Assinatura	Papel
inserir	(chave: str, valor: Any) -> None	Insere ou atualiza o valor associado a uma chave
buscar	(chave: str) -> Optional[Any]	Retorna o valor associado à chave, ou None se não existir
remover	(chave: str) -> bool	Remove a chave; retorna True/False indicando se ela existia
existe	(chave: str) -> bool	Verifica presença de uma chave, sem retornar o valor
todos_valores	() -> List[Any]	Retorna todos os valores armazenados, usado pelas operações de listagem/pesquisa/filtragem (Seção 6.2 e 6.3)
__len__	() -> int	Permite len(estrutura), usado por SistemaMissoes.quantidade_corpos()

Estrutura implementada: Tabela Hash
Local: src/estruturas/tabela_hash.py

Hash Implementação:

- Encadeamento externo: cada posição do vetor de buckets é uma lista de pares (chave, valor), nenhuma estrutura nativa de alto nível é usada para o armazenamento chave-valor.
- Função de hash própria, aplicando módulo a cada passo para evitar overflow:

```
h = 0
for caractere in chave:
    h = (h * 31 + ord(caractere)) % capacidade
```

- Redimensionamento dinâmico: quando o fator de carga ultrapassa 0.7, a capacidade dobra e todos os elementos são reinseridos (_redimensionar).

Estrutura implementada: Árvore B

Local: src/estruturas/arvore_b.py

- Árvore B de grau m (padrão 3): cada nó guarda no máximo 2t - 1 chaves e, se for interno, no máximo 2t filhos. As chaves de cada nó ficam sempre ordenadas, e filhos[i] contém chaves menores que chaves[i] e filhos[i+1] contém chaves maiores. Nenhuma estrutura pronta de biblioteca é usada, apenas listas para guardar chaves, valores e filhos dentro de cada nó.
- Chave do sistema: a distância ao corpo central (km). Como vários corpos podem ter a mesma distância, cada chave guarda uma lista de valores (valores[i]), e uma chave repetida só acrescenta o corpo à lista, sem criar nó novo.
- Inserção com divisão preventiva: a descida até a folha divide todo filho cheio que encontra no caminho (_dividir_filho), de modo que nunca é preciso voltar para cima. A divisão gera dois nós com t - 1 chaves e sobe a chave do meio para o pai:

Estrutura implementada: Trie

Local: src/estruturas/trie.py

Trie Implementação:

- Árvore de prefixos: cada nó guarda um dicionário de filhos (letra → nó), um marcador fim_de_palavra e o valor associado (o corpo celeste). Cada caractere do nome corresponde a um nível da árvore, e nomes com o mesmo início compartilham o mesmo caminho a partir da raiz.
- Chave do sistema: o nome do corpo celeste. Antes de inserir ou buscar, a chave é normalizada para minúsculas (_normalizar), então a busca ignora maiúsculas e minúsculas.
- Inserção: percorre os caracteres do nome criando os nós que faltam. Se o nome já existe, apenas atualiza o valor e não aumenta a quantidade de elementos.
- Busca exata (buscar, existe): desce a árvore seguindo os caracteres (_encontrar_no) e só retorna o valor se o nó final estiver marcado como fim de palavra. O custo depende do tamanho do nome, não da quantidade de corpos.
- Busca por prefixo (buscar_por_prefixo): desce até o nó do prefixo e coleta todos os valores da subárvore. A coleta é iterativa, com pilha explícita, e visita os filhos em ordem alfabética, então o resultado já sai ordenado por nome. Um prefixo vazio devolve todos os corpos:

Instrumentação obrigatória:

Exposta ao usuário pela opção 9 do menu (imprimir_estatisticas_hash), que imprime capacidade atual, elementos armazenados, fator de carga, total de colisões e total de redimensionamentos.

Métrica	Onde é medida
total_colisoes	Incrementada em inserir() sempre que o bucket de destino já tem algum elemento, e também durante _redimensionar(), ao reposicionar os elementos na nova tabela
total_redimensionamentos	Incrementada a cada _redimensionar()
fator_carga	Propriedade calculada (quantidade_elementos / capacidade)

4. Funcionalidades do Sistema

Local: src/sistema.py

Todas as operações de consulta e filtragem são expostas ao usuário pelo menu em main.py (opções 3 a 9), que apenas coleta a entrada e delega para SistemaMissoes, nunca acessando TabelaHash diretamente.

Consulta de elementos:

- buscar_por_id(id_corpo: str) -> Optional[CorpoCeleste] (opção 3 do menu).

  Localiza um corpo celeste específico a partir do seu identificador único (ex: "terre", "mars"), delegando diretamente para TabelaHash.buscar(). É a única operação de consulta que usa a estrutura de dados avançada em busca exata, complexidade **O(1) em média**, já que a Tabela Hash foi escolhida justamente por resolver esse tipo de acesso eficientemente. No pior caso teórico (todas as chaves colidindo no mesmo bucket), a busca degrada para **O(n)**, mas o redimensionamento dinâmico (fator de carga máximo 0.7) mantém isso improvável na prática

Pesquisa por atributos:

- pesquisar_por_nome_parcial(trecho: str) -> List[CorpoCeleste]: retorna todos os corpos cujo nome começa com o trecho informado (case-insensitive). Usa a Trie, indexada pelo nome do corpo: a busca desce a árvore seguindo os caracteres do trecho (O(m), com m = tamanho do trecho) e coleta apenas os valores da subárvore daquele prefixo. O custo é O(m + N), onde N é o número de nós da subárvore, ou seja, depende só do tamanho do resultado e não da quantidade total de corpos carregados. Como os filhos são visitados em ordem alfabética, o resultado já sai ordenado por nome.
- pesquisar_por_tipo(tipo: str) -> List[CorpoCeleste]: retorna todos os corpos de um bodyType específico (Planet, Moon, Dwarf Planet, Asteroid, Comet). Continua sendo uma busca linear O(n) sobre _tabela.todos_valores(), porque nenhuma das três estruturas é indexada por tipo: a Tabela Hash só acelera a

busca exata pela chave de indexação (id), a Trie é indexada por nome e a Árvore B por distância. Essa limitação é conhecida e documentada.

Listagem e filtragem:

- listar_todos() -> List[CorpoCeleste]: retorna todos os corpos carregados, sem nenhum critério de filtro, a partir da Tabela Hash. O(n), em ordem não definida.
- filtrar_por_faixa_distancia(minimo_km, maximo_km) -> List[CorpoCeleste]: retorna os corpos cuja distância ao Sol está dentro da faixa [minimo, maximo] informada pelo usuário. Usa a Árvore B, indexada por distancia_km: o percurso entra apenas nos filhos que podem conter chaves dentro da faixa, em O(log n + k), onde k é o tamanho do resultado, em vez de inspecionar todos os elementos (O(n)). Como a Árvore B mantém as chaves ordenadas, o resultado já sai em ordem crescente de distância. Corpos com a mesma distância ficam na mesma chave e são todos devolvidos.
- listar_ordenado_por_distancia() -> List[CorpoCeleste]: retorna todos os corpos ordenados crescentemente por distância ao Sol, por percurso em ordem da Árvore B (todos_valores). Como a estrutura já mantém as chaves ordenadas, não é necessário chamar uma função de ordenação: o custo é O(n), contra O(n log n) da abordagem anterior com sorted().

Todos os corpos são indexados nas três estruturas no momento do carregamento dos dados (método _indexar), então a Tabela Hash, a Trie e a Árvore B sempre armazenam o mesmo conjunto. As buscas por nome e por faixa de distância alimentam as estatísticas de nós visitados da Trie e da Árvore B (opções 10 e 11 do menu), o que serve de evidência de que as estruturas estão sendo usadas pelas funcionalidades do sistema.

5. Análise de complexidade amortizada - Rehashing da Tabela Hash

O problema com o pior caso "ingênuo":

Isoladamente, uma chamada a _redimensionar() custa O(n), pois todos os n elementos armazenados são percorridos e reinseridos na tabela nova. Se essa análise fosse aplicada ingenuamente a cada inserção, seria fácil concluir, de forma excessivamente pessimista, que toda inserção custa O(n) no pior caso. Isso não reflete o comportamento real do sistema ao longo de uma sequência de n inserções.

Método contábil:

Cobra-se de cada operação inserir() um custo amortizado constante, maior que seu custo real médio, e o excedente é guardado como "crédito" para pagar os redimensionamentos futuros.

- Seja a capacidade sempre dobrada quando o fator de carga ultrapassa 0.7 (_fator_carga_maximo).
- Custo real de uma inserção sem colisão de redimensionamento: O(1) (cálculo do hash + append no bucket).
- Cobrando 3 unidades de crédito por inserção:
  - 1 unidade paga a própria inserção;
  - 2 unidades ficam depositadas no elemento recém-inserido.
- Quando o redimensionamento ocorre, cada um dos n elementos que precisam ser reinseridos já acumulou 2 créditos, exatamente o suficiente para pagar seu próprio recálculo de hash e reinserção na tabela nova (custo real O(1) por elemento reinserido).

Como nenhuma operação nunca fica com saldo de crédito negativo, o custo amortizado por inserção é O(1), mesmo que o custo de uma operação isolada de redimensionamento seja O(n).

6. Estratégia Gulosa - Planejamento e Triagem de Missões

Local: src/missao/planejador_rota.py

Partindo do Sol (posição 0 no eixo de distância), o sistema monta uma rota que maximiza o potencial científico total da missão, respeitando duas restrições simultâneas de recurso:

Restrição	Significado
combustível_milhões_km	Orçamento total de combustível, expresso como a distância (em milhões de km) que a nave consegue percorrer. É consumido ao longo da missão e não é reabastecido.
custo_parada_milhoes_km	Combustível fixo gasto a cada parada (por exemplo, para entrar em órbita), na mesma unidade. O valor padrão é 0.

A nave nunca retorna a uma posição anterior (viagem estritamente progressiva, a favor da distância crescente ao Sol). Cada passo até um corpo consome:

custo_do_passo = distancia_do_trecho + custo_de_parada

Critério de seleção dos candidatos:

O benefício de cada corpo é o seu potencial científico, definido como o quanto sua densidade se desvia da densidade de referência (a da Terra, ≈5514 kg/m³):

potencial_cientifico = | densidade_corpo - 5514 |

Corpos muito mais densos (núcleo metálico) ou muito menos densos (composição gelada/gasosa) que a Terra são tratados como mais interessantes para investigação. O custo de cada candidato é custo do trecho até ele + o custo da parada, a partir da posição atual da nave. A cada iteração, dentre os candidatos ainda não visitados, à frente da posição atual e alcançáveis com o combustível restante (custo_do_passo ≤ combustivel_restante), o algoritmo escolhe o que maximiza a razão:

razão = potencial_cientifico / custo_do_passo

Corpos sem massa ou raio suficientes para calcular densidade são descartados dos candidatos (descartados_sem_dados). Luas também são excluídas, pois seu campo de distância é relativo ao planeta que orbitam, não ao Sol, e misturá-lo na mesma régua geraria posições sem sentido físico.

Estratégia implementada:

```
posição_atual = 0 (Sol)
combustível_restante = combustivel_milhoes_km

enquanto combustível_restante > 0:
    para cada candidato não visitado à frente de posição_atual:
        trecho = distância(candidato) - posição_atual
        custo = trecho + custo_de_parada
        se custo > combustível_restante: descarta (fora de alcance)
        senão: calcula razão = potencial(candidato) / custo
    se nenhum candidato alcançável: encerra a missão
    senão:
        escolhe o candidato de maior razão
        posição_atual = distância(candidato escolhido)
        combustível_restante = combustível_restante - custo
```

A cada iteração, o algoritmo faz uma escolha local e definitiva (não há backtracking): decide o próximo destino olhando somente para o benefício imediato do próximo passo, sem considerar o impacto dessa escolha nos passos futuros.

Exemplo de execução:

Com um conjunto pequeno (Mercúrio, Vênus, Terra, Marte, Ceres e Júpiter), combustível de 10.000 milhões de km e custo de parada de 1.000 milhões de km:

Passo	Posição da nave (milhões de km)	Escolhido	Trecho (milhões de km)	Passo (milhões de km)	Razão (potencial/km)
1	0	Ceres	413,7	1.413,7	0,00000242 (Júpiter ficou em 0,00000235 e Marte em 0,00000129)
2	413,7	Júpiter	364,8	1.364,8	0,00000307 (único candidato à frente)
3	778,5	nenhum	-	-	-

Rota final: Ceres -> Júpiter, potencial científico total de ≈7.605, com 2.778,5 milhões de km de combustível gastos e 7.221,5 milhões de km de sobra. Mercúrio, Vênus e Terra ficam de fora por terem potencial científico baixo (a Terra tem potencial zero). Marte é ultrapassado: a partir do Sol, Ceres tinha razão maior, e como a nave nunca retorna, Marte deixa de ser alcançável.

Justificativa do critério:
A razão potencial/custo (em vez do potencial absoluto) evita que o algoritmo sempre prefira o corpo cientificamente "mais exótico" independentemente do custo. Ela avalia o benefício pelo combustível realmente gasto para alcançá-lo.
O custo fixo de parada é necessário para que o problema tenha sentido. Como a nave só avança, a soma dos trechos é sempre igual à posição da última parada, de modo que, sem esse custo, parar em corpos intermediários seria gratuito e o ótimo seria trivial (visitar todos os corpos dentro do alcance). Com o custo fixo, cada parada consome combustível, e o algoritmo passa a pular corpos de baixo potencial. Um custo de parada pequeno aproxima o comportamento do caso sem custo, um custo grande favorece poucas paradas em corpos de potencial alto.
A escolha de densidade como fator de potencial científico se justifica porque a maioria dos corpos da API tem massa e raio registrados, então a densidade é calculável para muitos corpos, o que cobre uma faixa muito maior de candidatos.

Limitações da estratégia gulosa:

A escolha estritamente gulosa não garante a rota de potencial científico total máximo (ótimo global), por alguns motivos:

- **Sem visão de futuro:** o algoritmo escolhe, a cada passo, a melhor razão imediata, sem considerar se essa escolha vai "fechar o caminho" para destinos que, somados, rendem mais. No exemplo acima, Marte foi ignorado porque Ceres tinha razão maior a partir do Sol. Visitar Marte, Ceres e Júpiter custaria apenas 3.000 milhões de km em paradas mais 778,5 de distância, bem dentro do orçamento, e renderia um potencial total maior que o da rota escolhida (≈9.185 contra ≈7.605).
- **Decisões irreversíveis:** uma vez escolhido um destino, não há como o algoritmo voltar atrás caso, poucos passos depois, perceba que a decisão anterior impediu uma rota melhor. É uma característica de qualquer algoritmo guloso.
- **Sensibilidade ao custo de parada:** a rota depende do valor escolhido para custo_parada_milhoes_km, que é um parâmetro da missão e não um dado dos corpos.

Encontrar a rota de potencial máximo garantido exigiria explorar todos os subconjuntos possíveis de paradas (ou usar programação dinâmica sobre a posição e o combustível restante), o que teria custo computacional muito maior. A abordagem gulosa resolve o problema em tempo proporcional a paradas × número_de_candidatos, ou seja, no pior caso $O(n^2)$.

7. Programação Dinâmica - Missão das Luas

Local: src/missao/planejador_luas.py

O sistema responde a uma pergunta de planejamento: dado um número alvo de luas, quais planetas a missão deve visitar para chegar a **exatamente** esse número, passando pelo **menor número possível de planetas**? O problema é uma variante do problema do troco, com a seguinte correspondência:

Troco	Missão das Luas
Moeda	Planeta
Valor da moeda	Número de luas do planeta (quantidade_luas)
Valor a pagar	Número alvo de luas (x)
Mínimo de moedas	Mínimo de planetas visitados

Candidatos:
Entram na análise os corpos com e_planeta verdadeiro e quantidade_luas maior que zero (selecionar_planetas). Planetas sem luas nunca ajudam a atingir o alvo e são descartados. Luas e planetas anões ficam de fora, pois não são planetas. O número de luas vem do campo moons da API (len(moons)), então depende do que a API retornou na data da consulta.

Formulação:
Seja dp[i][s] o melhor resultado usando apenas os i primeiros planetas candidatos para somar exatamente s luas. Cada célula guarda uma tupla (número de planetas, soma das distâncias), e as tuplas são comparadas em ordem lexicográfica: o número de planetas decide primeiro, e a distância só entra em caso de empate.

dp[0][0] = (0, 0), dp[0][s>0] = (inviável)
dp[i][s] = min(dp[i-1][s], dp[i-1][s - m_i] + (1, dist_i)), se s >= m_i

onde m_i é o número de luas do planeta i e dist_i é a sua distância ao Sol. A primeira opção não visita o planeta i, e a segunda visita. A célula dp[n][x] guarda a resposta. Se ela for inviável, nenhuma combinação de planetas soma exatamente x luas.

Estratégia implementada:

```
dp[0][0] = (0, 0); todas as outras células = inviável

para cada planeta i = 1..n:

  para cada s = 0..x:

    melhor = dp[i-1][s]                    # não visita o planeta i

    se s >= luas(i) e dp[i-1][s - luas(i)] é viável:

      candidato = dp[i-1][s - luas(i)] + (1, dist(i))

      se candidato < melhor: melhor = candidato

    dp[i][s] = melhor

se dp[n][x] é inviável: sem solução

senão: reconstrói de trás para frente

  s = x

  para i = n..1:

    se dp[i][s] != dp[i-1][s]:             # o planeta i foi usado

      guarda o planeta i; s = s - luas(i)
```

Ao contrário do algoritmo guloso, a tabela guarda o melhor resultado de **todos** os subproblemas, então a solução final considera todas as combinações possíveis sem enumerá-las uma a uma. A reconstrução percorre a tabela de trás para frente: se o valor da célula mudou ao incluir o planeta i, esse planeta faz parte da solução.

Para x = 3, a resposta é Terra + Marte: 2 planetas, 3 luas, 377,5 milhões de km de soma de distâncias. Resultados para outros alvos com os mesmos quatro planetas:

Justificativa da formulação:

A abordagem gulosa não serve para esse problema. Escolher sempre o planeta com mais luas pode levar a um beco sem saída, mesmo existindo solução. Exemplo (valores ilustrativos): com planetas de 5, 4 e 3 luas e x = 7, o guloso pega o de 5 luas e não consegue completar o restante (5 + 3 = 8 e 5 + 4 = 9), mas a programação dinâmica encontra 4 + 3. A DP resolve isso porque decide cada planeta olhando para todas as somas possíveis. A formulação do troco 0/1 é coerente com o contexto da aplicação, pois cada planeta é visitado no máximo uma vez, e atende ao requisito de decisão combinatória do enunciado, já que a escolha é de um subconjunto e não uma simples ordenação.

O desempate pela distância ao Sol representa o menor gasto de combustível para chegar aos planetas. Ele torna o problema mais rico do que o troco puro, mas é só uma estimativa: o custo real depende da rota percorrida e da ordem das visitas.

Complexidade:

- Tempo: $O(n \cdot x)$, com n = número de planetas candidatos e x = número alvo de luas, pois a tabela tem $(n + 1) \cdot (x + 1)$ células e cada uma é preenchida em tempo constante.
- Espaço: $O(n \cdot x)$, para guardar a tabela inteira, necessária para reconstruir a solução. É possível reduzir para $O(x)$ com um vetor unidimensional, mas a reconstrução exigiria guardar as escolhas à parte.
- Reconstrução: $O(n)$.

8. Fluxo máximo - Edmonds-Karp

Local: src/missao/fluxo_maximo.py

O sistema responde a uma pergunta de logística: dada uma origem e um destino, qual é a maior quantidade de carga (toneladas por mês) que a rede de rotas consegue transportar entre os dois astros, usando todas as rotas ao mesmo tempo? Este é o problema clássico de **fluxo máximo**, resolvido com o algoritmo de **Edmonds-Karp**, que é o método de Ford-Fulkerson com busca em largura (BFS) para escolher os caminhos.

Modelagem do problema

O grafo não é montado dentro do módulo de fluxo. Ele é importado de grafo_sistema_solar.py, o mesmo usado pelo A*, de modo que todos os planejadores trabalham sobre a mesma rede.

Elemento do grafo	Significado na aplicação
Vértice	Astro (Terra, Marte, Júpiter, ...)
Aresta	Rota espacial entre dois astros
Capacidade	Toneladas de carga por mês que a rota suporta
Fonte (origem)	Astro de onde a carga parte
Sorvedouro (destino)	Astro para onde a carga deve chegar

Cada rota guarda três atributos: preço (mil créditos), tempo (dias) e capacidade (toneladas/mês). O fluxo máximo usa apenas a capacidade. As rotas são de mão dupla, e **cada sentido tem a capacidade da rota**. Em montar_grafo, cada aresta é inserida nas listas de adjacência dos dois astros. A rede tem 14 astros e 19 rotas.

Estruturas de dados

Estrutura	Papel
residual[u][v]	Capacidade que ainda sobra de u para v. Começa igual à capacidade original e é atualizada a cada caminho enviado
original[u][v]	Cópia das capacidades iniciais, usada no final para calcular o fluxo que passou em cada rota
deque (fila)	Fila da BFS e da busca do corte mínimo
ResultadoFluxo	Resultado: valor do fluxo, caminhos aumentantes, rotas com fluxo, corte mínimo e a melhor rota única

Estratégia implementada:

```
residual = capacidades originais do grafo
fluxo_total = 0
repita:
   caminho = BFS(residual, origem, destino)      # só usa arestas com residual > 0
   se não há caminho: pare
   gargalo = menor residual ao longo do caminho
   para cada aresta (u, v) do caminho:
      residual[u][v] -= gargalo                  # consome capacidade
      residual[v][u] += gargalo                  # permite desfazer a decisão depois
   fluxo_total += gargalo
```

Três pontos explicam o funcionamento:

- **Caminho aumentante:** é um caminho da origem ao destino em que todas as arestas ainda têm capacidade livre. O fluxo enviado em cada iteração é o gargalo, ou seja, a menor capacidade residual do caminho.
- **Aresta reversa:** ao enviar fluxo de u para v, aumenta-se também o residual de v para u. Isso permite que iterações futuras cancelem parte de um envio anterior. Por isso o algoritmo não é guloso: uma má escolha de caminho pode ser corrigida depois.
- **BFS (a diferença para o Ford-Fulkerson genérico):** a busca em largura sempre encontra o caminho com **menos arestas**. Isso limita o número de iterações a $O(V \cdot E)$, independentemente do valor das capacidades. O Ford-Fulkerson com DFS pode depender do valor do fluxo e ficar muito lento com capacidades grandes.

O algoritmo termina quando a BFS não consegue mais alcançar o destino. O fluxo acumulado é então o máximo possível.

Justificativa da formulação:

- **Por que fluxo máximo:** a pergunta da agência é de capacidade da rede, não de custo de um único trajeto. Dijkstra e A* (que minimizam preço ou tempo) respondem “qual o melhor caminho”; o fluxo máximo responde “quanto a rede inteira consegue transportar”. Os dois usam o mesmo grafo, mas com atributos diferentes.
- **Por que BFS:** garante terminação em número de iterações polinomial, independente das capacidades, e produz caminhos curtos, que são mais fáceis de interpretar como rotas reais.
- **Por que mão dupla com capacidade por sentido:** reflete a descrição do grafo, em que a capacidade vale nos dois sentidos, e é modelado com duas entradas no residual, uma para cada sentido.

Complexidade:

- **Tempo:** $O(V \cdot E^2)$. A BFS custa $O(E)$, e o número de caminhos aumentantes é no máximo $O(V \cdot E)$, pois cada gargalo satura pelo menos uma aresta e a distância (em arestas) até o destino nunca diminui. Na rede do projeto, com $V = 14$ e $E = 19$, isso é desprezível.
- **Espaço:** $O(V + E)$ para o grafo residual, a cópia das capacidades e a fila.
- **Corte mínimo:** $O(V + E)$, por uma única BFS adicional.
- **Caminho de maior capacidade:** $O(E \log V)$, por ser um Dijkstra com heap.

9. Busca A* - Volta para a Terra

Local: src/missao/busca_a_estrela.py (o grafo vem de grafo_sistema_solar.py)

O sistema responde a uma pergunta de rota: partindo de qualquer astro da rede, qual é o melhor caminho para voltar à Terra? Como cada trecho tem dois custos, **preço** (mil créditos) e **tempo** (dias), o usuário escolhe qual deles minimizar. O problema é resolvido com a **busca A***, que combina o custo já gasto com uma estimativa do custo que falta, e por isso expande bem menos astros que uma busca cega.

Modelagem do problema

O grafo é importado de grafo_sistema_solar.py, o mesmo usado pelo fluxo máximo, então os dois planejadores trabalham sobre a mesma rede com atributos diferentes.

Elemento da busca	Significado na aplicação
Estado	Astro onde a nave se encontra
Estado inicial	Astro escolhido pelo usuário
Estado objetivo	Terra (constante OBJETIVO)
Ação	Viajar por uma rota até um astro vizinho
Custo do passo	Preço ou tempo da rota, conforme o critério escolhido
Critério	"preco" ou "tempo" (constante CRITERIOS)

Cada aresta guarda preço, tempo e capacidade. O A* usa apenas os dois primeiros e ignora a capacidade, que só interessa ao fluxo máximo. As rotas são de mão dupla, então o custo de ida e volta é o mesmo. Esta parte também não depende da API: a rede de rotas e seus custos são definidos pelo projeto.

Estruturas de dados:

Estrutura	Papel
No	Nó da busca: guarda o astro (estado), o nó anterior (pai) e o preço e o tempo acumulados até ali
fronteira (heap)	Fila de prioridade com os nós a expandir, ordenada por f = g + h
contador	Desempate no heap: com f igual, sai primeiro o nó inserido antes. Também evita comparar objetos No
g_na_fronteira	Melhor custo g conhecido para cada astro que está na fronteira
explorados	Conjunto de astros já expandidos
heuristicas	Tabela de h(astro) para cada critério

O No acumula **os dois custos ao mesmo tempo**. A busca só decide pelo critério escolhido, mas ao chegar à Terra o resultado informa também o preço e o tempo totais da rota. Isso permite comparar o que se paga em um critério ao otimizar o outro. O caminho é reconstruído seguindo os ponteiros pai até a origem e invertendo a sequência.

Função heurística:

A heurística estima quanto falta para chegar à Terra a partir de cada astro. Ela é construída em duas etapas:

1. **Distância até a Terra** (distancia_terra): a diferença entre a distância do astro ao Sol e a da Terra, em milhões de km. Europa e Titan recebem o mesmo valor de Júpiter e Saturno, já que orbitam esses planetas.
2. **Fator de conversão:** essa distância é multiplicada pelo **menor custo por milhão de km** encontrado entre todas as arestas.

```
h_preco(astro) = distancia_terra(astro) × 2,0    # mil créditos por milhão de km

h_tempo(astro) = distancia_terra(astro) × 0,25   # dias por milhão de km
```

Usar o menor custo por km é o que garante que a estimativa nunca passe do custo real, isto é, que a heurística seja **admissível**. Ela também é **consistente**: a distância até a Terra é uma diferença de posições numa mesma reta, então h nunca diminui mais do que o custo da aresta percorrida. Por isso cada astro é expandido no máximo uma vez, e o primeiro caminho que chega à Terra já é o ótimo.

A estimativa é bem justa. Em Júpiter, por exemplo, o h_preco é 1.257,8 contra um custo real mínimo de 1.378, e o h_tempo é 157,2 contra um tempo real mínimo de 160. Uma heurística próxima do custo real direciona bem a busca.

Estratégia implementada:

```
fronteira = { inicio com f = h(inicio) }

g_na_fronteira[inicio] = 0

explorados = {}

enquanto a fronteira não está vazia:

    no = nó de menor f = g + h

    se no.estado já foi explorado: ignora          # entrada antiga do heap

    marca no.estado como explorado

    se no.estado == Terra: devolve caminho, custos e estatísticas

    para cada vizinho do astro (preço, tempo da rota):

        se vizinho já foi explorado: ignora

        novo_g = custo acumulado do critério + custo da rota

        se vizinho está na fronteira com g melhor ou igual: ignora

        insere o novo nó com f = novo_g + h(vizinho)

        g_na_fronteira[vizinho] = novo_g

fronteira esvaziou: não existe rota até a Terra
```

Resultado e estatísticas

A busca devolve um dicionário com a rota, o custo no critério escolhido e os totais de preço e tempo. Ele traz também duas métricas de desempenho:

Campo	Significado
expandidos	Quantos astros foram expandidos até chegar à Terra
fronteira_max	Maior número de astros distintos esperando na fronteira

Elas medem o esforço da busca e permitem comparar o A* com a versão sem heurística. Se a Terra não for alcançável, caminho volta como None e imprimir_resultado avisa que não existe rota.

Exemplo de execução: Partindo de Júpiter

Critério minimizado	Rota	Preço total	Tempo total
Preço	Júpiter → Ceres → Marte → Terra	1.378 mil créditos	323 dias
Tempo	Júpiter → Terra	2.052 mil créditos	160 dias